Idenso

Symbolic tensor identities for Spenso and Symbolica

1 Overview

Idenso is the symbolic identity and simplification layer for tensors encoded in the form that Spenso can parse from Symbolica expressions. It provides representation symbols, index tooling, reversible "cooking" of subexpressions, selective expansion, and identities for Dirac, metric, epsilon, and color algebra.

> **Symbolic, not component expansion**
> Idenso rewrites abstract tensor expressions. Functions such as `expand_mink` distribute factorized terms that carry selected index families; they do not turn every tensor into a dense array of components. Concrete tensor storage and network execution belong to Spenso.

1.1 A controlled rewrite pipeline

There is no universally correct "simplify everything" order. A robust workflow makes each phase explicit:

- initialize the standard representations and tensor symbols;
- inspect dangling indices and normalize or wrap dummy-index namespaces when combining expressions;
- expand only the sectors needed by the next identity pass;
- simplify gamma, metric, or color structures as appropriate;
- canonicalize and compare results using the conventions of the consuming calculation.

Expansion can grow an expression quickly, so perform it as late and selectively as possible. Wrapping indices is especially important before multiplying independently constructed expressions: equal printed index names can otherwise acquire an unintended contraction.

1.2 Representations and cooking

The representation layer defines spin-fundamental, color-fundamental, color-sextet, bispinor, and color-adjoint types together with their duality. `initialize()` forces the related Symbolica symbols and Spenso tags to be registered before parsing or rewriting expressions.

The `Cookable` API can replace selected function-like subexpressions or index payloads with compact symbols and later reverse the operation. Use reversible settings when the original expression must be recovered. A cooked symbol is an intermediate encoding, not a portable serialized physics result unless the associated mapping and settings are retained.

Idenso is consumed by GammaLoop and its tensor conventions are shared with Spenso. Start with the Idenso README, then consult the Rust or Python API reference for supported functions, signatures, and feature gates.

2 Tutorial

This tutorial uses Idenso's Python community module to contract one metric tensor with a vector. It is intentionally small: the point is to establish the registered tensor syntax and observe one algebra pass before composing a larger Dirac or color pipeline.

2.1 Prerequisites

Idenso is mounted at `symbolica.community.idenso`; it is not installed by `pip install idenso`. Use a Symbolica Python distribution whose release includes the Idenso and Spenso community modules, or build those modules through the `symbolica-community` assembly. Verify the actual environment first:

The assembly and installation instructions live in `symbolica-community`.

```
python -c "import symbolica.community.idenso; import symbolica.community.spenso"
```

If that fails, install or rebuild the community assembly before continuing. Generating a `.pyi` file does not mount the native module.

2.2 Simplify one metric contraction

Save the following as `metric_first.py`:

```
from symbolica.community.idenso import initialize, list_dangling, simplify_metrics
from symbolica.community.spenso import Representation, TensorName

initialize()
rep = Representation.euc(3)
mu = rep("mu")
nu = rep("nu")

g = TensorName.g()
q = TensorName("q")
expression = g(mu, nu) * q(mu)

print("free before:", list_dangling(expression))
reduced = simplify_metrics(expression)
print("reduced:", reduced)
print("free after:", list_dangling(reduced))
```

Run `python metric_first.py`. Success means the metric is removed from the reduced expression, the contracted `mu` no longer appears as a free index, and the result is a rank-one expression carrying `nu`. Printed output can vary with the installed Symbolica version; inspect the expression structure instead of comparing exact text.

> **Initialize before parsing or rewriting**
> `initialize()` installs Idenso's representation and tensor symbols. Calling it explicitly at the start of a standalone script makes registration order clear and keeps expressions in the Spenso-compatible form that Idenso's matchers expect.

2.3 Grow this into an algebra pipeline

Keep transformations observable while developing:

- call `list_dangling` before and after a pass to catch accidental contractions;
- use `wrap_dummies` before multiplying expressions built in independent index namespaces;
- expand only the Minkowski, bispinor, or color sector needed by the next pass;
- apply `simplify_metrics`, `simplify_gamma`, and `simplify_color` as distinct phases;
- canonicalize only after the physics-specific identities required by the calculation are explicit.

2.4 Troubleshooting and next steps

- `ModuleNotFoundError` for `symbolica.community.idenso` means the installed Symbolica build does not include the native module; a `.pyi` type stub or source checkout alone is not enough.

- If the metric remains unchanged, construct its slots with the same `Representation` and abstract index objects as the vector. Plain Symbolica functions do not automatically carry Spenso tensor structure.
- If a free index disappears unexpectedly, inspect repeated names and use `wrap_dummies` before combining separately constructed expressions.
- Continue with the syntax-and-algebra manual, then use the Python and Rust API references for signatures and feature gates.

3 Syntax, indices, and algebra passes

Idenso consumes the Symbolica function form emitted for Spenso structures. Function heads carry representation meaning and their arguments carry abstract indices or tensor data. Initialize Idenso before parsing so Symbolica attributes, Spenso tags, and dual representations are all registered in one deterministic order.

3.1 Indices and cooking

Free indices describe the result; repeated compatible indices describe contractions. Before combining independently built expressions, wrap or rename dummy-index namespaces so equal printed names do not create accidental contractions. Cooking temporarily replaces selected index payloads or function subexpressions with compact symbols. Retain the generated map and settings whenever uncooking is required.

Cooking is especially useful before expensive canonicalization, but it changes what downstream matchers can see. Uncook before applying an identity whose pattern depends on the hidden tensor head or index structure.

3.2 Metric and epsilon operations

Metric contraction raises, lowers, or identifies compatible Lorentz indices according to the registered representation. Epsilon identities depend on dimension, ordering, and sign conventions; expand them only when the next pass benefits from the larger expression. Keep Minkowski expansion selective to avoid distributing unrelated scalar factors.

3.3 Dirac and color algebra

Dirac passes simplify gamma chains, traces, slashes, and spinor-compatible contractions. Color passes handle fundamental/adjoint deltas, generators, structure constants, and registered group parameters. Apply one algebra family at a time and inspect the intermediate expression; an all-at-once fixed-point loop can obscure which convention produced a sign or normalization.

> **Canonical does not mean physically equivalent by itself**
> Canonical ordering makes structurally equivalent expressions comparable under the registered rules. On-shell relations, gauge choices, dimension-specific identities, and model parameter substitutions must still be requested explicitly.

3.4 Schoonschip network parsing

The Schoonschip path parses large gamma-chain expressions into the same Spenso-compatible network model before contraction. It resolves aliases and normalizes the expression before constructing the network. Spenso then plans and executes contractions, while Idenso supplies the representation and algebra rules. Avoid substituting scalar parameters too early: doing so can substantially increase intermediate expression size even when the resulting tensor network is unchanged.

Concrete syntax and rewrite cases are documented in the Spenso/Symbolica syntax note and the color and Dirac comparison. The Schoonschip parsing guide shows how normalization, network construction, and contraction fit together.

Tensor storage and execution remain owned by Spenso.

4 Rust and Python APIs

4.1 Rust package

The `idenso` crate exposes representation types and syntax macros together with several rewrite families:

- `IndexTooling` covers canonicalization, conjugation, index wrapping, and dangling-index inspection for Symbolica atoms;
- `Cookable`, `CookSettings`, and the cook filters control reversible or flattening encodings;
- `SelectiveExpand` expands terms by recognized representation families;
- `dirac`, `color`, `epsilon`, and shorthand modules implement algebra-specific rewrites;
- `representations::initialize` installs the standard representation and tensor symbols.

Representation helper macros such as `bis!`, `cof!`, and `coad!` construct the symbolic forms expected by Spenso and Idenso. The Rust API reference gives their accepted forms and the return types of each rewrite. APIs behind `bincode`, `reference-cases`, `python`, and `python_stubgen` are available only when the matching Cargo feature is enabled.

4.2 Python community module

> **Part of Symbolica community**
> Import Idenso from `symbolica.community.idenso`. The `idenso` Cargo package supplies this community module when built with its Python feature; it is not a standalone PyPI distribution and is not included in the GammaLoop wheel merely because the crates share a repository.

Install a Symbolica Python distribution only if its release manifest says it includes Idenso, then verify the actual assembly with `python -c "import symbolica.community.idenso"`. There is no `pip install idenso` fallback. For a source build, add this crate to the external symbolica-community assembly with the `python` feature and call `IdensoModule`'s `SymbolicaCommunityModule` registration when that extension is assembled. Building the Rust crate alone does not add the community module to an already installed Symbolica package.

The Python API operates on Symbolica expressions and groups naturally into four phases:

- setup: `initialize`;
- expansion: `expand_bis`, `expand_mink`, `expand_mink_bis`, `expand_metrics`, and `expand_color`;
- index preparation: `wrap_indices`, `wrap_dummies`, `list_dangling`, `cook_indices`, and `cook_function`;
- algebra: `dirac_adjoint`, `simplify_gamma`, `to_dots`, `simplify_metrics`, and `simplify_color`.

```python
from symbolica.community.idenso import (
    initialize,
    list_dangling,
    simplify_metrics,
)

initialize()
# `expr` is a Symbolica Expression written with Spenso-compatible tensor forms.
external_indices = list_dangling(expr)
reduced = simplify_metrics(expr)
```

The example deliberately leaves expression construction to Symbolica and Spenso: Idenso does not define a second parser syntax. Apply one transformation at a time while developing a pipeline so expression growth and convention changes remain observable.

For implementation details, start with the Rust API and the Python binding.

5 Releases and change history

> **Version coverage**
> This version of Idenso is 0.3.0, but the published changelog currently ends at 0.2.5. Release notes for 0.3.0 are not yet available, so the history below is incomplete for that version.

When upgrading Idenso, pay particular attention to representation initialization, dummy-index handling, canonicalization, and Dirac, metric, and color conventions. A rewrite change can alter the shape or ordering of a symbolic result without changing the mathematical intent, so callers that persist or compare printed expressions should verify their chosen canonical form.

For reproducible upgrades, record the Idenso version and enabled features with the calculation. Do not assume that changes between 0.2.5 and 0.3.0 are fully represented in the available notes.

Read the available release notes at crates/idenso/CHANGELOG.md.

API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

`idenso`

Idenso's supported representation and algebra API. The manual owns the representation conventions; this scope provides *stable* source links.

`Index tooling`

```
Adds canonicalization, wrapping, conjugation, and dangling-index inspection to
Symbolica atoms.
```

```
Defines operations related to manipulating abstract indices within symbolic
expressions,
particularly relevant for physics calculations involving tensor structures and
diagrams.

This trait provides methods for conjugating expressions, wrapping indices, and
identifying
external ("dangling") indices.
```

```
#[doc =
" Defines operations related to manipulating abstract indices within symbolic
expressions,"]
#[doc =
" particularly relevant for physics calculations involving tensor structures and
diagrams."]
#[doc = ""]
#[doc =
" This trait provides methods for conjugating expressions, wrapping indices, and
identifying"]
#[doc = " external (\"dangling\") indices."] pub trait IndexTooling
{
    fn canonize < Aind : AbsInd + ParseableAind + DummyAind >
    (& self, new_dummy : impl FnMut(usize) -> Aind,) -> Atom;
    #[doc =
    " Wraps all abstract indices within the expression using a specified header
symbol."]
    #[doc = ""]
    #[doc =
    " This transforms indices like `mink(dim,idx)` into `mink(dim,header(idx))`.
Useful for distinguishing"]
    #[doc =
    " between different copies of an expression, e.g., an amplitude and its complex
conjugate."]
    #[doc = ""] #[doc = " # Arguments"]
    #[doc =
    " * `header` - The [`Symbol`] to use as the wrapping function name."]
    #[doc = ""] #[doc = " # Returns"]
    #[doc = " A new [`Atom`] with all indices wrapped."] fn
    wrap_indices(& self, header : Symbol) -> Atom; fn spenso_conj(& self) ->
    Atom; fn spenso_print < 'a > (& 'a self, settings : & SpensoPrintSettings)
    -> AtomPrinter < 'a > ;
    #[doc =
    " Wraps only the dummy (contracted) abstract indices within the expression using
a header symbol."]
    #[doc = ""]
```

```
    #[doc =
    " Identifies indices that appear contracted (e.g., one covariant, one
contravariant)"]
    #[doc =
    " and wraps only those, leaving external indices unchanged. Transforms `idx ->
header(idx)`"]
    #[doc = " for dummy indices `idx`."] #[doc = ""] #[doc = " # Arguments"]
    #[doc =
    " * `header` - The [`Symbol`] to use as the wrapping function name for dummy
indices."]
    #[doc = ""] #[doc = " # Returns"]
    #[doc = " A new [`Atom`] with only dummy indices wrapped."] fn
    wrap_dummies < Aind : AbsInd + DummyAind + ParseableAind >
    (& self, header : Symbol) -> Atom;
    #[doc = " Computes the physics-aware conjugate of the expression."]
    #[doc = ""]
    #[doc =
    " Applies conjugation rules specific to physics objects like spinors, gamma
matrices,"]
    #[doc =
    " color representations, and the imaginary unit `i`. See implementation details
for"]
    #[doc = " specific rules applied."] #[doc = ""] #[doc = " # Returns"]
    #[doc = " A new [`Atom`] representing the conjugated expression."] fn
    dirac_adjoint < Aind : DummyAind + ParseableAind + AbsInd > (& self) ->
    eyre :: Result < Atom > ; fn
    conjugate_transpose(& self, rep : impl RepName) -> Atom;
    #[doc =
    " Identifies and returns a list of dangling (external, uncontracted) indices."]
    #[doc = ""]
    #[doc =
    " Analyzes the expression to find indices that are not summed over. Returns
them"]
    #[doc =
    " as `Atom`s. Note that dual indices might be represented wrapped in a `dind`
function."]
    #[doc = ""] #[doc = " # Returns"]
    #[doc = " A `Vec<Atom>` where each `Atom` represents a dangling index."]
    fn list_dangling < Aind : AbsInd + DummyAind + ParseableAind > (& self) ->
    Vec < Atom > ; fn alias_subtensors(& self, tensor_name : & str) ->
    AliasedAtom;
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_index_tools<T: idenso::IndexTooling>(_expression: &T) {}
```

`canonize`

Kind: `Method`

```
fn canonize < Aind : AbsInd + ParseableAind + DummyAind >
(& self, new_dummy : impl FnMut(usize) -> Aind,) -> Atom;
```

`wrap_indices`

Kind: `Method`

```
#[doc =
" Wraps all abstract indices within the expression using a specified header symbol."]
#[doc = ""]
#[doc =
" This transforms indices like `mink(dim,idx)` into `mink(dim,header(idx))`. Useful
for distinguishing"]
#[doc =
" between different copies of an expression, e.g., an amplitude and its complex
conjugate."]
#[doc = ""] #[doc = " # Arguments"]
#[doc = " * `header` - The [`Symbol`] to use as the wrapping function name."]
#[doc = ""] #[doc = " # Returns"]
#[doc = " A new [`Atom`] with all indices wrapped."] fn
wrap_indices(& self, header : Symbol) -> Atom;
```

Wraps all abstract indices within the expression using a specified header symbol.

This transforms indices like `mink(dim,idx)` into `mink(dim,header(idx))`. Useful for distinguishing
between different copies of an expression, e.g., an amplitude and its complex conjugate.

Arguments
* `header` - The [`Symbol`] to use as the wrapping function name.

Returns
A new [`Atom`] with all indices wrapped.

spenso_conj

Kind: Method

```
fn spenso_conj(& self) -> Atom;
```

spenso_print

Kind: Method

```
fn spenso_print < 'a > (& 'a self, settings : & SpensoPrintSettings) ->
AtomPrinter < 'a > ;
```

wrap_dummies

Kind: Method

```
#[doc =
" Wraps only the dummy (contracted) abstract indices within the expression using a
header symbol."]
#[doc = ""]
#[doc =
" Identifies indices that appear contracted (e.g., one covariant, one
contravariant)"]
#[doc =
" and wraps only those, leaving external indices unchanged. Transforms `idx ->
header(idx)`"]
#[doc = " for dummy indices `idx`."] #[doc = ""] #[doc = " # Arguments"]
#[doc =
" * `header` - The [`Symbol`] to use as the wrapping function name for dummy
indices."]
#[doc = ""] #[doc = " # Returns"]
```

```
#[doc = " A new [`Atom`] with only dummy indices wrapped."] fn wrap_dummies <
Aind : AbsInd + DummyAind + ParseableAind > (& self, header : Symbol) -> Atom;
```

Wraps only the dummy (contracted) abstract indices within the expression using a header symbol.

Identifies indices that appear contracted (e.g., one covariant, one contravariant) and wraps only those, leaving external indices unchanged. Transforms `idx -> header(idx)`
for dummy indices `idx`.

Arguments
* `header` - The [`Symbol`] to use as the wrapping function name for dummy indices.

Returns
A new [`Atom`] with only dummy indices wrapped.

dirac_adjoint

Kind: Method

```
#[doc = " Computes the physics-aware conjugate of the expression."]
#[doc = ""]
#[doc =
" Applies conjugation rules specific to physics objects like spinors, gamma
matrices,"]
#[doc =
" color representations, and the imaginary unit `i`. See implementation details for"]
#[doc = " specific rules applied."] #[doc = ""] #[doc = " # Returns"]
#[doc = " A new [`Atom`] representing the conjugated expression."] fn
dirac_adjoint < Aind : DummyAind + ParseableAind + AbsInd > (& self) -> eyre
:: Result < Atom > ;
```

Computes the physics-aware conjugate of the expression.

Applies conjugation rules specific to physics objects like spinors, gamma matrices, color representations, and the imaginary unit `i`. See implementation details for specific rules applied.

Returns
A new [`Atom`] representing the conjugated expression.

conjugate_transpose

Kind: Method

```
fn conjugate_transpose(& self, rep : impl RepName) -> Atom;
```

list_dangling

Kind: Method

```
#[doc =
" Identifies and returns a list of dangling (external, uncontracted) indices."]
#[doc = ""]
#[doc =
" Analyzes the expression to find indices that are not summed over. Returns them"]
#[doc =
" as `Atom`s. Note that dual indices might be represented wrapped in a `dind`
function."]
#[doc = ""] #[doc = " # Returns"]
#[doc = " A `Vec<Atom>` where each `Atom` represents a dangling index."] fn
```

```
list_dangling < Aind : AbsInd + DummyAind + ParseableAind > (& self) -> Vec <
Atom > ;
```

```
Identifies and returns a list of dangling (external, uncontracted) indices.

Analyzes the expression to find indices that are not summed over. Returns them
as `Atom`s. Note that dual indices might be represented wrapped in a `dind` function.

# Returns
A `Vec<Atom>` where each `Atom` represents a dangling index.
```

alias_subtensors

Kind: `Method`

```
fn alias_subtensors(& self, tensor_name : & str) -> AliasedAtom;
```

Exhaustive reference · Source

Index cooking settings

```
Controls reversible, flattened, and filtered encodings of tensor indices.

Settings for cooking functions and representation-slot indices.

Tags are attached through `SymbolBuilder::new(...).with_tags(...)`, so they must be
fully namespaced Symbolica tags such as `idenso::cooked`.
#[doc = " Settings for cooking functions and representation-slot indices."]
#[doc = ""]
#[doc =
" Tags are attached through `SymbolBuilder::new(...).with_tags(...)`, so they must
be"]
#[doc = " fully namespaced Symbolica tags such as `idenso::cooked`."]
#[derive(Debug, Clone, PartialEq, Eq)] pub struct CookSettings
{ mode : CookMode, source : CookSourceFilter, output_tags : CookOutputTags, }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let settings = idenso::CookSettings::reversible();
```

mode

Kind: `Field`

```
CookMode
```

source

Kind: `Field`

```
CookSourceFilter
```

output_tags

Kind: `Field`

```
CookOutputTags
```

Exhaustive reference · Source

Cookable expressions

```
Applies or reverses index-cooking transformations on symbolic expressions.
```

Convenience methods for cooking Symbolica atoms.

```
#[doc = " Convenience methods for cooking Symbolica atoms."] pub trait
Cookable
{
    #[doc =
    " Reversibly cook function-like subexpressions with the default cooked tag."]
    fn cook(& self) -> Atom;
    #[doc = " Cook function-like subexpressions with explicit settings."] fn
    cook_with_settings(& self, settings : & CookSettings) -> Atom;
    #[doc = " Undo default reversible cooking."] fn uncook(& self) -> Atom;
    #[doc = " Undo reversible cooking that used explicit settings."] fn
    uncook_with_settings(& self, settings : & CookSettings) -> Atom;
    #[doc =
    " Flatten cook abstract-index payloads in recognized representation slots."]
    fn cook_indices(& self) -> Atom;
    #[doc =
    " Cook representation-slot index payloads with explicit settings."] fn
    cook_indices_with_settings(& self, settings : & CookSettings) -> Atom;
    #[doc = " Flatten cook a single atom into one symbol."] fn
    cook_function(& self) -> Result < Atom, CookingError > ;
    #[doc = " Cook a single atom into one symbol with explicit settings."] fn
    cook_function_with_settings(& self, settings : & CookSettings) -> Result <
    Atom, CookingError > ;
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_cookable<T: idenso::Cookable>(_expression: &T) {}
```

`cook`

Kind: `Method`

```
#[doc =
" Reversibly cook function-like subexpressions with the default cooked tag."]
fn cook(& self) -> Atom;
```

Reversibly cook function-like subexpressions with the default cooked tag.

`cook_with_settings`

Kind: `Method`

```
#[doc = " Cook function-like subexpressions with explicit settings."] fn
cook_with_settings(& self, settings : & CookSettings) -> Atom;
```

Cook function-like subexpressions with explicit settings.

`uncook`

Kind: `Method`

```
#[doc = " Undo default reversible cooking."] fn uncook(& self) -> Atom;
```

Undo default reversible cooking.

`uncook_with_settings`

Kind: `Method`

```
#[doc = " Undo reversible cooking that used explicit settings."] fn
uncook_with_settings(& self, settings : & CookSettings) -> Atom;
```

Undo reversible cooking that used explicit settings.

cook_indices

Kind: Method

```
#[doc =
" Flatten cook abstract-index payloads in recognized representation slots."]
fn cook_indices(& self) -> Atom;
```

Flatten cook abstract-index payloads in recognized representation slots.

cook_indices_with_settings

Kind: Method

```
#[doc = " Cook representation-slot index payloads with explicit settings."] fn
cook_indices_with_settings(& self, settings : & CookSettings) -> Atom;
```

Cook representation-slot index payloads with explicit settings.

cook_function

Kind: Method

```
#[doc = " Flatten cook a single atom into one symbol."] fn
cook_function(& self) -> Result < Atom, CookingError > ;
```

Flatten cook a single atom into one symbol.

cook_function_with_settings

Kind: Method

```
#[doc = " Cook a single atom into one symbol with explicit settings."] fn
cook_function_with_settings(& self, settings : & CookSettings) -> Result <
Atom, CookingError > ;
```

Cook a single atom into one symbol with explicit settings.

Exhaustive reference · Source

Selective expansion

Expands only tensor families selected by their representation.

Expands only tensor families selected by their representation.

```
pub trait SelectiveExpand
{
    fn expand_in_patterns(& self, pats : & [Pattern]) -> Vec < (Atom, Atom) >
    ; fn expand_metrics(& self) -> Vec < (Atom, Atom) >
    {
        let metric_pat = function! (ETS.metric, RS.a__).to_pattern(); let
        id_pat = function! (ETS.metric, RS.a__).to_pattern();
        self.expand_in_patterns(& [metric_pat, id_pat])
    } fn expand_bis(& self) -> Vec < (Atom, Atom) >
    {
        let bis = Bispinor {}; let bis_pat = function!
        (RS.f_, RS.a___, bis.to_symbolic([RS.b__]), RS.c___).to_pattern();
        self.expand_in_patterns(& [bis_pat])
    } fn expand_mink(& self) -> Vec < (Atom, Atom) >
    {
        let mink = Minkowski {}; let mink_pat = function!
```

```
        (RS.f_, RS.a___, mink.to_symbolic([RS.b__]), RS.c___).to_pattern();
        self.expand_in_patterns(& [mink_pat])
    } fn expand_mink_bis(& self) -> Vec < (Atom, Atom) >
    {
        let mink = Minkowski {}; let mink_pat = function!
        (RS.f_, RS.a___, mink.to_symbolic([RS.b__]), RS.c___).to_pattern();
        let bis = Bispinor {}; let bis_pat = function!
        (RS.f_, RS.a___, bis.to_symbolic([RS.b__]), RS.c___).to_pattern();
        self.expand_in_patterns(& [mink_pat, bis_pat])
    }
}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_expansion<T: idenso::selective_expand::SelectiveExpand>(_expression: &T)
{}
```

`expand_in_patterns`

Kind: `Method`

```
fn expand_in_patterns(& self, pats : & [Pattern]) -> Vec < (Atom, Atom) > ;
```

`expand_metrics`

Kind: `Method`

```
fn expand_metrics(& self) -> Vec < (Atom, Atom) >
{
    let metric_pat = function! (ETS.metric, RS.a__).to_pattern(); let id_pat =
    function! (ETS.metric, RS.a__).to_pattern();
    self.expand_in_patterns(& [metric_pat, id_pat])
}
```

`expand_bis`

Kind: `Method`

```
fn expand_bis(& self) -> Vec < (Atom, Atom) >
{
    let bis = Bispinor {}; let bis_pat = function!
    (RS.f_, RS.a___, bis.to_symbolic([RS.b__]), RS.c___).to_pattern();
    self.expand_in_patterns(& [bis_pat])
}
```

`expand_mink`

Kind: `Method`

```
fn expand_mink(& self) -> Vec < (Atom, Atom) >
{
    let mink = Minkowski {}; let mink_pat = function!
    (RS.f_, RS.a___, mink.to_symbolic([RS.b__]), RS.c___).to_pattern();
    self.expand_in_patterns(& [mink_pat])
}
```

`expand_mink_bis`

Kind: `Method`

```
fn expand_mink_bis(& self) -> Vec < (Atom, Atom) >
{
    let mink = Minkowski {}; let mink_pat = function!
    (RS.f_, RS.a___, mink.to_symbolic([RS.b__]), RS.c___).to_pattern(); let
    bis = Bispinor {}; let bis_pat = function!
    (RS.f_, RS.a___, bis.to_symbolic([RS.b__]), RS.c___).to_pattern();
    self.expand_in_patterns(& [mink_pat, bis_pat])
}
```

Exhaustive reference · Source

`Bispinor representation macro`

`Builds stripped or indexed bispinor representation atoms in Spenso syntax.`

`Builds a stripped or indexed bispinor representation atom.`

```
#[doc = " Builds a stripped or indexed bispinor representation atom."]
#[macro_export] macro_rules! bis
{
    ($($args:tt)*) =>
    {
        spenso::spenso_rep_atom!($crate::representations::Bispinor {};
        $($args)*)
    };
}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::bis!(4);
```

Exhaustive reference · Source

`Color-fundamental macro`

`Builds stripped or indexed color-fundamental representation atoms.`

`Builds a stripped or indexed color-fundamental representation atom.`

```
#[doc =
" Builds a stripped or indexed color-fundamental representation atom."]
#[macro_export] macro_rules! cof
{
    ($($args:tt)*) =>
    {
        spenso::spenso_rep_atom!($crate::representations::ColorFundamental {};
        $($args)*)
    };
}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::cof!(Nc);
```

Exhaustive reference · Source

`Color-adjoint representation macro`

`Builds stripped or indexed color-adjoint representation atoms.`

```
Builds a stripped or indexed color-adjoint representation atom.
```

```
#[doc = " Builds a stripped or indexed color-adjoint representation atom."]
#[macro_export] macro_rules! coad
{
    ($($args:tt)*) =>
    {
        spenso::spenso_rep_atom!($crate::representations::ColorAdjoint {};
        $($args)*)
    };
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::coad!(Na);
```

Exhaustive reference · Source

`Dirac gamma macro`

```
Builds a Dirac gamma chain factor or an explicitly indexed gamma tensor.
```

```
Builds a gamma matrix.

With one argument, this builds a chain factor using the placeholder indices
`in` and `out`; use this form only as a factor inside `chain!` or `trace!`.
With three arguments, this builds the ordinary gamma tensor with explicit
spinor endpoints and a Lorentz slot.

Arguments are converted through `spenso::shadowing::IntoAtom`, so they
can be typed slots, atoms, or atom views.

# Examples

```ignore
use idenso::gamma;
use spenso::{chain, slot};

let factor = gamma!(slot!(mink4, mu));
let chain_expr = chain!(slot!(bis4, a), slot!(bis4, b), factor);
let default_tensor = gamma!(mu, a, b);
let indexed_tensor = gamma!(1, 2, 3);
let pattern_tensor = gamma!(RS.a__, RS.b__, RS.c__);
let pattern_tensor_with_slots = gamma!(RS.a__, [RS.d_, RS.i_], [RS.d_, RS.j_]);
let mixed_tensor = gamma!(mu, slot!(bis_d, a), 1);
let explicit_tensor = gamma!(slot!(mink_d, mu), slot!(bis_d, a), slot!(bis_d, b));
```
```

```
#[doc = " Builds a gamma matrix."] #[doc = ""]
#[doc =
" With one argument, this builds a chain factor using the placeholder indices"]
#[doc =
" `in` and `out`; use this form only as a factor inside `chain!` or `trace!`."]
#[doc =
" With three arguments, this builds the ordinary gamma tensor with explicit"]
#[doc = " spinor endpoints and a Lorentz slot."] #[doc = ""]
#[doc =
" Arguments are converted through `spenso::shadowing::IntoAtom`, so they"]
```

```
#[doc = " can be typed slots, atoms, or atom views."] #[doc = ""]
#[doc = " # Examples"] #[doc = ""] #[doc = " ```ignore"]
#[doc = " use idenso::gamma;"] #[doc = " use spenso::{chain, slot};"]
#[doc = ""] #[doc = " let factor = gamma!(slot!(mink4, mu));"]
#[doc = " let chain_expr = chain!(slot!(bis4, a), slot!(bis4, b), factor);"]
#[doc = " let default_tensor = gamma!(mu, a, b);"]
#[doc = " let indexed_tensor = gamma!(1, 2, 3);"]
#[doc = " let pattern_tensor = gamma!(RS.a__, RS.b__, RS.c__);"]
#[doc =
" let pattern_tensor_with_slots = gamma!(RS.a__, [RS.d_, RS.i_], [RS.d_, RS.j_]);"]
#[doc = " let mixed_tensor = gamma!(mu, slot!(bis_d, a), 1);"]
#[doc =
" let explicit_tensor = gamma!(slot!(mink_d, mu), slot!(bis_d, a), slot!(bis_d,
b));"]
#[doc = " ```"] #[macro_export] macro_rules! gamma
{
    ($mu:expr) =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma).add_arg(symbolica::atom::Atom::var(
    }; ($base:ident. $mu:ident, $($rest:tt)*) =>
    {
        $crate::gamma!(@tensor
spenso::structure::representation::RepName::to_symbolic(&spenso::structure::representation::Minkows
        {}, [$base.$mu],); $($rest)*)
    }; ($mu:ident, $($rest:tt)*) =>
    {{
            let mink =
spenso::structure::representation::RepName::new_rep(&spenso::structure::representation::Minkowski
            {}, 4,);
            $crate::gamma!(@tensor spenso::slot!(mink, $mu); $($rest)*)
        }}; ($mu:literal, $($rest:tt)*) =>
    {{
            let mink =
spenso::structure::representation::RepName::new_rep(&spenso::structure::representation::Minkowski
            {}, 4,);
            $crate::gamma!(@tensor spenso::slot!(mink, $mu); $($rest)*)
        }}; ($mu:expr, $($rest:tt)*) =>
    { $crate::gamma!(@tensor $mu; $($rest)*) };
    (@tensor $mu:expr; [$($i:expr),+ $(,)?], $($rest:tt)*) =>
    {
        $crate::gamma!(@tensor2 $mu,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$($i),+],); $($rest)*)
    }; (@tensor $mu:expr; $base:ident. $i:ident, $($rest:tt)*) =>
    {
        $crate::gamma!(@tensor2 $mu,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$i],); $($rest)*)
    }; (@tensor $mu:expr; $i:ident, $($rest:tt)*) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
```

```
            $crate::gamma!(@tensor2 $mu, spenso::slot!(bis, $i); $($rest)*)
        }}; (@tensor $mu:expr; $i:literal, $($rest:tt)*) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
            $crate::gamma!(@tensor2 $mu, spenso::slot!(bis, $i); $($rest)*)
        }}; (@tensor $mu:expr; $i:expr, $($rest:tt)*) =>
    { $crate::gamma!(@tensor2 $mu, $i; $($rest)*) };
    (@tensor2 $mu:expr, $i:expr; [$($j:expr),+ $(,)?]) =>
    {
        $crate::gamma!(@tensor_done $mu, $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$($j),+],))
    }; (@tensor2 $mu:expr, $i:expr; $base:ident. $j:ident) =>
    {
        $crate::gamma!(@tensor_done $mu, $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$j],))
    }; (@tensor2 $mu:expr, $i:expr; $j:ident) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
            $crate::gamma!(@tensor_done $mu, $i, spenso::slot!(bis, $j))
        }}; (@tensor2 $mu:expr, $i:expr; $j:literal) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
            $crate::gamma!(@tensor_done $mu, $i, spenso::slot!(bis, $j))
        }}; (@tensor2 $mu:expr, $i:expr; $j:expr) =>
    { $crate::gamma!(@tensor_done $mu, $i, $j) };
    (@tensor_done $mu:expr, $i:expr, $j:expr) =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma).add_arg(spenso::shadowing::IntoAtom
    };
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::gamma;
use spenso::{chain, slot};

let factor = gamma!(slot!(mink4, mu));
let chain_expr = chain!(slot!(bis4, a), slot!(bis4, b), factor);
let default_tensor = gamma!(mu, a, b);
let indexed_tensor = gamma!(1, 2, 3);
let pattern_tensor = gamma!(RS.a__, RS.b__, RS.c__);
let pattern_tensor_with_slots = gamma!(RS.a__, [RS.d_, RS.i_], [RS.d_, RS.j_]);
let mixed_tensor = gamma!(mu, slot!(bis_d, a), 1);
let explicit_tensor = gamma!(slot!(mink_d, mu), slot!(bis_d, a), slot!(bis_d, b));
```

Supported usage

```
let tensor = idenso::gamma!(1, 2, 3);
```

Exhaustive reference · Source

`Gamma-zero macro`

```
Builds a gamma-zero chain factor or an explicitly indexed gamma-zero tensor.

Builds a gamma-zero factor or tensor.

With no arguments, this builds a chain factor using the placeholder indices
`in` and `out`; use this form only as a factor inside `chain!` or `trace!`.
With two arguments, this builds the ordinary gamma-zero tensor between
explicit spinor endpoints.

Endpoints can be typed slots, atoms, atom views, default four-dimensional
bispinor identifiers/literals, pattern variables such as `RS.i__`, or
bracketed bispinor argument lists such as `[RS.d_, RS.i_]`.

# Examples

```ignore
use idenso::gamma0;

let factor = gamma0!();
let default_tensor = gamma0!(i, j);
let pattern_tensor = gamma0!(RS.i__, RS.j__);
let dimensioned_pattern_tensor = gamma0!([RS.d_, RS.i_], [RS.d_, RS.j_]);
```
#[doc = " Builds a gamma-zero factor or tensor."] #[doc = ""]
#[doc =
" With no arguments, this builds a chain factor using the placeholder indices"]
#[doc =
" `in` and `out`; use this form only as a factor inside `chain!` or `trace!`."]
#[doc =
" With two arguments, this builds the ordinary gamma-zero tensor between"]
#[doc = " explicit spinor endpoints."] #[doc = ""]
#[doc =
" Endpoints can be typed slots, atoms, atom views, default four-dimensional"]
#[doc =
" bispinor identifiers/literals, pattern variables such as `RS.i__`, or"]
#[doc = " bracketed bispinor argument lists such as `[RS.d_, RS.i_]`."]
#[doc = ""] #[doc = " # Examples"] #[doc = ""] #[doc = " ```ignore"]
#[doc = " use idenso::gamma0;"] #[doc = ""]
#[doc = " let factor = gamma0!();"]
#[doc = " let default_tensor = gamma0!(i, j);"]
#[doc = " let pattern_tensor = gamma0!(RS.i__, RS.j__);"]
#[doc =
" let dimensioned_pattern_tensor = gamma0!([RS.d_, RS.i_], [RS.d_, RS.j_]);"]
#[doc = " ```"] #[macro_export] macro_rules! gamma0
{
    () =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma0).add_arg(symbolica::atom::Atom::var
    }; ($base:ident. $i:ident, $($rest:tt)*) =>
    {
```

```
        $crate::gamma0!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$i],); $($rest)*)
    }; ([$($i:expr),+ $(,)?], $($rest:tt)*) =>
    {
        $crate::gamma0!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$($i),+],); $($rest)*)
    }; ($i:ident, $($rest:tt)*) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
            $crate::gamma0!(@tensor spenso::slot!(bis, $i); $($rest)*)
        }}; ($i:literal, $($rest:tt)*) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,);
            $crate::gamma0!(@tensor spenso::slot!(bis, $i); $($rest)*)
        }}; ($i:expr, $($rest:tt)*) =>
    { $crate::gamma0!(@tensor $i; $($rest)*) };
    (@tensor $i:expr; $base:ident. $j:ident) =>
    {
        $crate::gamma0!(@tensor_done $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$j],))
    }; (@tensor $i:expr; [$($j:expr),+ $(,)?]) =>
    {
        $crate::gamma0!(@tensor_done $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$($j),+],))
    }; (@tensor $i:expr; $j:ident) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,); $crate::gamma0!(@tensor_done $i, spenso::slot!(bis, $j))
        }}; (@tensor $i:expr; $j:literal) =>
    {{
            let bis =
spenso::structure::representation::RepName::new_rep(&$crate::representations::Bispinor
            {}, 4,); $crate::gamma0!(@tensor_done $i, spenso::slot!(bis, $j))
        }}; (@tensor $i:expr; $j:expr) =>
    { $crate::gamma0!(@tensor_done $i, $j) }; (@tensor_done $i:expr, $j:expr)
    =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma0).add_arg(spenso::shadowing::IntoAto
    };
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::gamma0;

let factor = gamma0!();
let default_tensor = gamma0!(i, j);
let pattern_tensor = gamma0!(RS.i__, RS.j__);
let dimensioned_pattern_tensor = gamma0!([RS.d_, RS.i_], [RS.d_, RS.j_]);
```

Supported usage

```
let tensor = idenso::gamma0!(1, 2);
```

Exhaustive reference · Source

`Gamma-five macro`

```
Builds a gamma-five chain factor or an explicitly indexed gamma-five tensor.

Builds a gamma-five factor or tensor.

With no arguments, this builds a chain factor using the placeholder indices
`in` and `out`; the surrounding chain or trace owns the physical endpoints.
With two arguments, this builds the ordinary gamma-five tensor between
explicit spinor endpoints.

Pattern variables such as `RS.i__` are wrapped as bispinor arguments.

#[doc = " Builds a gamma-five factor or tensor."] #[doc = ""]
#[doc =
" With no arguments, this builds a chain factor using the placeholder indices"]
#[doc =
" `in` and `out`; the surrounding chain or trace owns the physical endpoints."]
#[doc =
" With two arguments, this builds the ordinary gamma-five tensor between"]
#[doc = " explicit spinor endpoints."] #[doc = ""]
#[doc =
" Pattern variables such as `RS.i__` are wrapped as bispinor arguments."]
#[macro_export] macro_rules! gamma5
{
    () =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma5).add_arg(symbolica::atom::Atom::var
    }; ($base:ident. $i:ident, $($rest:tt)*) =>
    {
        $crate::gamma5!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$i],); $($rest)*)
    }; ($i:expr, $($rest:tt)*) => { $crate::gamma5!(@tensor $i; $($rest)*) };
    (@tensor $i:expr; $base:ident. $j:ident) =>
    {
        $crate::gamma5!(@tensor_done $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::Bispinor
        {}, [$base.$j],))
    }; (@tensor $i:expr; $j:expr) => { $crate::gamma5!(@tensor_done $i, $j) };
    (@tensor_done $i:expr, $j:expr) =>
    {
symbolica::atom::FunctionBuilder::new($crate::dirac::AGS.gamma5).add_arg(spenso::shadowing::IntoAto
```

```
    };
}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let tensor = idenso::gamma5!(1, 2);
```

Exhaustive reference · Source

`Levi-Civita tensor macro`

```
Builds an arbitrary-rank antisymmetric epsilon tensor from atom-like arguments.

Builds an antisymmetric epsilon tensor with arbitrary rank.

Arguments are converted through `spenso::shadowing::IntoAtom`, so tests
and rewrite code can pass slots, atoms, atom views, symbols, and integers
without spelling out the conversion.
#[doc = " Builds an antisymmetric epsilon tensor with arbitrary rank."]
#[doc = ""]
#[doc =
" Arguments are converted through `spenso::shadowing::IntoAtom`, so tests"]
#[doc =
" and rewrite code can pass slots, atoms, atom views, symbols, and integers"]
#[doc = " without spelling out the conversion."] #[macro_export] macro_rules!
epsilon
{
    (; $factors:expr $(,)?) =>
    { $crate::epsilon::epsilon(($factors).into_iter()) };
    ($($arg:expr),* $(,)?) =>
    {{
            let builder =
            symbolica::atom::FunctionBuilder::new(*$crate::epsilon::EPSILON_SYMBOL);
            $(let builder =
            builder.add_arg(spenso::shadowing::IntoAtom::into_atom($arg));)*
            builder.finish()
        }};
}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let tensor = idenso::epsilon!(1, 2, 3, 4);
```

Exhaustive reference · Source

`Color-generator macro`

```
Builds a color-generator chain factor or an explicitly indexed generator tensor.

Builds a color-generator atom.

With one argument, this builds a fundamental generator factor for use inside
`chain!` or `trace!`. The factor uses the chain placeholder indices `in` and
`out`; the surrounding chain or trace owns the physical endpoints.

With three arguments, this builds an explicit generator `t(a,i,j)`. The first
```

```
argument is an adjoint color index, the second is a fundamental color index,
and the third is an anti-fundamental color index.

Arguments can be typed slots, atoms, atom views, pattern variables such as
`RS.a__`, or bracketed representation argument lists such as
`[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so
local slot bindings and numeric labels are used as-is.

# Examples

```ignore
use idenso::color_t;
use spenso::{slot, trace};

let factor = color_t!(slot!(coad_na, a));
let expr = trace!(&cof_nc, factor);
let factor_from_binding = color_t!(slot!(coad_na, a));
let explicit_tensor = color_t!(slot!(coad_na, a), slot!(cof_nc, i), slot!
(cof_nc.dual(), j));
let pattern_tensor = color_t!(RS.a__, RS.i__, RS.j__);
let dimensioned_pattern = color_t!([CS.adj_, RS.a_], [CS.nc_, RS.i_], [CS.nc_,
RS.j_]);
```
#[doc = " Builds a color-generator atom."] #[doc = ""]
#[doc =
" With one argument, this builds a fundamental generator factor for use inside"]
#[doc =
" `chain!` or `trace!`. The factor uses the chain placeholder indices `in` and"]
#[doc = " `out`; the surrounding chain or trace owns the physical endpoints."]
#[doc = ""]
#[doc =
" With three arguments, this builds an explicit generator `t(a,i,j)`. The first"]
#[doc =
" argument is an adjoint color index, the second is a fundamental color index,"]
#[doc = " and the third is an anti-fundamental color index."] #[doc = ""]
#[doc =
" Arguments can be typed slots, atoms, atom views, pattern variables such as"]
#[doc = " `RS.a__`, or bracketed representation argument lists such as"]
#[doc =
" `[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so"]
#[doc = " local slot bindings and numeric labels are used as-is."] #[doc = ""]
#[doc = " # Examples"] #[doc = ""] #[doc = " ```ignore"]
#[doc = " use idenso::color_t;"] #[doc = " use spenso::{slot, trace};"]
#[doc = ""] #[doc = " let factor = color_t!(slot!(coad_na, a));"]
#[doc = " let expr = trace!(&cof_nc, factor);"]
#[doc = " let factor_from_binding = color_t!(slot!(coad_na, a));"]
#[doc =
" let explicit_tensor = color_t!(slot!(coad_na, a), slot!(cof_nc, i), slot!
(cof_nc.dual(), j));"]
#[doc = " let pattern_tensor = color_t!(RS.a__, RS.i__, RS.j__);"]
#[doc =
" let dimensioned_pattern = color_t!([CS.adj_, RS.a_], [CS.nc_, RS.i_], [CS.nc_,
RS.j_]);"]
#[doc = " ```"] #[macro_export] macro_rules! color_t
{
    ([$($a:expr),+ $(,)?], $($rest:tt)+) =>
```

```
    {
        $crate::color_t!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$($a),+],); $($rest)*)
    }; ($base:ident. $a:ident, $($rest:tt)+) =>
    {
        $crate::color_t!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$base.$a],); $($rest)*)
    }; ($a:expr, $($rest:tt)+) => { $crate::color_t!(@tensor $a; $($rest)*) };
    ([$($a:expr),+ $(,)?] $(,)?) =>
    {
$crate::color::CS.chain_t(spenso::structure::representation::RepName::to_symbolic(&$crate::represen
        {}, [$($a),+],))
    }; ($base:ident. $a:ident $(,)?) =>
    {
$crate::color::CS.chain_t(spenso::structure::representation::RepName::to_symbolic(&$crate::represen
        {}, [$base.$a],))
    }; ($a:expr $(,)?) =>
    { $crate::color::CS.chain_t(spenso::shadowing::IntoAtom::into_atom($a)) };
    (@tensor $a:expr; [$($i:expr),+ $(,)?], $($rest:tt)*) =>
    {
        $crate::color_t!(@tensor2 $a,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorFundamental
        {}, [$($i),+],); $($rest)*)
    }; (@tensor $a:expr; $base:ident. $i:ident, $($rest:tt)*) =>
    {
        $crate::color_t!(@tensor2 $a,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorFundamental
        {}, [$base.$i],); $($rest)*)
    }; (@tensor $a:expr; $i:expr, $($rest:tt)*) =>
    { $crate::color_t!(@tensor2 $a, $i; $($rest)*) };
    (@tensor2 $a:expr, $i:expr; [$($j:expr),+ $(,)?] $(,)?) =>
    {
        $crate::color_t!(@tensor_done $a, $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAntiFundamen
        {}, [$($j),+],))
    }; (@tensor2 $a:expr, $i:expr; $base:ident. $j:ident $(,)?) =>
    {
        $crate::color_t!(@tensor_done $a, $i,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAntiFundamen
        {}, [$base.$j],))
    }; (@tensor2 $a:expr, $i:expr; $j:expr $(,)?) =>
    { $crate::color_t!(@tensor_done $a, $i, $j) };
    (@tensor_done $a:expr, $i:expr, $j:expr) =>
    {
        $crate::color::CS.explicit_t(spenso::shadowing::IntoAtom::into_atom($a),
        spenso::shadowing::IntoAtom::into_atom($i),
        spenso::shadowing::IntoAtom::into_atom($j),)
    };
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::color_t;
use spenso::{slot, trace};

let factor = color_t!(slot!(coad_na, a));
let expr = trace!(&cof_nc, factor);
let factor_from_binding = color_t!(slot!(coad_na, a));
let explicit_tensor = color_t!(slot!(coad_na, a), slot!(cof_nc, i), slot!
(cof_nc.dual(), j));
let pattern_tensor = color_t!(RS.a__, RS.i__, RS.j__);
let dimensioned_pattern = color_t!([CS.adj_, RS.a_], [CS.nc_, RS.i_], [CS.nc_,
RS.j_]);
```

Supported usage

```
let tensor = idenso::color_t!(1, 2, 3);
```

Exhaustive reference · Source

`Color structure-constant macro`

```
Builds an explicitly indexed adjoint color structure-constant tensor.

Builds an adjoint color structure-constant atom `f(a,b,c)`.

Arguments can be typed slots, atoms, atom views, pattern variables such as
`RS.a__`, or bracketed adjoint representation argument lists such as
`[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so
local slot bindings and numeric labels are used as-is.

# Examples

```ignore
use idenso::color_f;
use spenso::slot;

let expr = color_f!(slot!(coad_na, a), slot!(coad_na, b), slot!(coad_na, c));
let numeric_label_tensor = color_f!(1, 2, 3);
let pattern_tensor = color_f!(RS.a__, RS.b__, RS.c__);
let dimensioned_pattern = color_f!([CS.adj_, RS.a_], [CS.adj_, RS.b_], [CS.adj_,
RS.c_]);
```

#[doc = " Builds an adjoint color structure-constant atom `f(a,b,c)`."]
#[doc = ""]
#[doc =
" Arguments can be typed slots, atoms, atom views, pattern variables such as"]
#[doc =
" `RS.a__`, or bracketed adjoint representation argument lists such as"]
#[doc =
" `[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so"]
#[doc = " local slot bindings and numeric labels are used as-is."] #[doc = ""]
#[doc = " # Examples"] #[doc = ""] #[doc = " ```ignore"]
#[doc = " use idenso::color_f;"] #[doc = " use spenso::slot;"] #[doc = ""]
#[doc =
" let expr = color_f!(slot!(coad_na, a), slot!(coad_na, b), slot!(coad_na, c));"]
```

```
#[doc = " let numeric_label_tensor = color_f!(1, 2, 3);"]
#[doc = " let pattern_tensor = color_f!(RS.a__, RS.b__, RS.c__);"]
#[doc =
" let dimensioned_pattern = color_f!([CS.adj_, RS.a_], [CS.adj_, RS.b_], [CS.adj_,
RS.c_]);"]
#[doc = " ```"] #[macro_export] macro_rules! color_f
{
    ([$($a:expr),+ $(,)?], $($rest:tt)+) =>
    {
        $crate::color_f!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$($a),+],); $($rest)*)
    }; ($base:ident. $a:ident, $($rest:tt)+) =>
    {
        $crate::color_f!(@tensor
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$base.$a],); $($rest)*)
    }; ($a:expr, $($rest:tt)+) => { $crate::color_f!(@tensor $a; $($rest)*) };
    (@tensor $a:expr; [$($b:expr),+ $(,)?], $($rest:tt)*) =>
    {
        $crate::color_f!(@tensor2 $a,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$($b),+],); $($rest)*)
    }; (@tensor $a:expr; $base:ident. $b:ident, $($rest:tt)*) =>
    {
        $crate::color_f!(@tensor2 $a,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$base.$b],); $($rest)*)
    }; (@tensor $a:expr; $b:expr, $($rest:tt)*) =>
    { $crate::color_f!(@tensor2 $a, $b; $($rest)*) };
    (@tensor2 $a:expr, $b:expr; [$($c:expr),+ $(,)?] $(,)?) =>
    {
        $crate::color_f!(@tensor_done $a, $b,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$($c),+],))
    }; (@tensor2 $a:expr, $b:expr; $base:ident. $c:ident $(,)?) =>
    {
        $crate::color_f!(@tensor_done $a, $b,
spenso::structure::representation::RepName::to_symbolic(&$crate::representations::ColorAdjoint
        {}, [$base.$c],))
    }; (@tensor2 $a:expr, $b:expr; $c:expr $(,)?) =>
    { $crate::color_f!(@tensor_done $a, $b, $c) };
    (@tensor_done $a:expr, $b:expr, $c:expr) =>
    {
        $crate::color::CS.structure_f(spenso::shadowing::IntoAtom::into_atom($a),
        spenso::shadowing::IntoAtom::into_atom($b),
        spenso::shadowing::IntoAtom::into_atom($c),)
    };
}
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::color_f;
use spenso::slot;

let expr = color_f!(slot!(coad_na, a), slot!(coad_na, b), slot!(coad_na, c));
let numeric_label_tensor = color_f!(1, 2, 3);
let pattern_tensor = color_f!(RS.a__, RS.b__, RS.c__);
let dimensioned_pattern = color_f!([CS.adj_, RS.a_], [CS.adj_, RS.b_], [CS.adj_,
RS.c_]);
```

Supported usage

```
let tensor = idenso::color_f!(1, 2, 3);
```

Exhaustive reference · Source

`Initialize representations`

```
Initializes Idenso's standard representations and tensor symbols.
```

```
Register Idenso's built-in representations and algebra symbols with Symbolica.

Symbolica calls this during community-module initialization. Calling it again
is safe and ensures the standard Lorentz, spinor, and color objects exist.
```

```
fn initialize()
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Supported usage

```
idenso::representations::initialize();
```

Exhaustive reference · Source

`idenso-community`

```
Exports registered by idenso.
```

`cook_function`

```
Convert a single function call into a flattened variable symbol.
```

```
Convert a single function call into a flattened variable symbol.

Transforms a function expression with arguments into a single symbolic variable
whose name encodes both the function name and its arguments. This is the
atomic version of `cook_indices()`, operating on individual function calls
rather than complete expressions.

**Function Cooking Transform:**
- Simple function: `f(a, b)` → `f_a_b`
- Nested arguments: `tensor(rep(mu))` → `tensor_rep_mu`
- Multiple arguments: `gamma(mu, alpha, beta)` → `gamma_mu_alpha_beta`
- Complex names: `my_function(x, y)` → `my_function_x_y`


**Constraints:**
- Input must be a single function call (not sum, product, etc.)
- Arguments must be cookable (symbols, numbers, simple functions)
- Cannot cook expressions containing polynomials or complex structures

# Arguments
```

```
- `self_`: expression representing a single function call to cook

# Returns
Expression containing the flattened variable symbol.

# Raises
`TypeError` if input is not a cookable function or contains invalid argument types.

# Examples:
```python
import symbolica as sp
from symbolica.community.idenso import cook_function

# Simple function cooking
f = sp.S('f')
a, b = sp.S('a','b')

cooked = cook_function(f(a, b))
print(cooked)  # Outputs: f_a_b
```

def cook_function(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import cook_function
help(cook_function)
```

Exhaustive reference · Source

`cook_indices`

Convert complex nested index structures into flattened symbolic names.

```
Convert complex nested index structures into flattened symbolic names.

Transforms hierarchical index expressions within tensor function arguments
into simplified, flat symbolic representations. This "cooking" process is
essential for pattern matching, simplification, and computational efficiency
when dealing with complex tensor expressions.

**Index Cooking Transformation:**
- Nested structure: `mink(4, f(g(h(μ))))` → `mink(4, f_g_h_mu)`
- Function chains: `lorentz(up(mu))` → `lorentz(up_mu)`
- Complex arguments: `tensor(rep(dim,type(idx)))` → `tensor(rep(dim,type_idx))`

**Scope:**
- Only affects indices appearing as function arguments
- Preserves top-level function structure
# Arguments
- `self_`: expression containing complex nested index structures

# Returns
Expression with flattened, simplified index names.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
```
```

```
import symbolica as sp
from symbolica.community.idenso import wrap_indices, cook_indices

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, x)  # T(mu, nu; x)
print(tensor_with_args)
print(
cook_indices(wrap_indices(tensor_with_args.to_expression(), sp.S("wrap")))
)
```

```
def cook_indices(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import cook_indices
help(cook_indices)
```

Exhaustive reference · Source

`dirac_adjoint`

```
Return the Dirac adjoint of a Symbolica tensor expression.
```

```
Return the Dirac adjoint of a Symbolica tensor expression.

This reverses the spinor chain and applies the representation-aware adjoint
rules used by Idenso's Dirac algebra.
```

```
def dirac_adjoint(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import dirac_adjoint
help(dirac_adjoint)
```

Exhaustive reference · Source

`expand_bis`

```
Expands factorized terms containing Dirac bispinor indices.
```

```
Expands factorized terms containing Dirac bispinor indices.

Finds and expands factorized expressions involving bispinor tensors and spinors,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

**Factorization Expansion:**
- `ψ(α) * (γ(μ,α,β) + σ(μ,α,β)) → ψ(α)*γ(μ,α,β) + ψ(α)*σ(μ,α,β)`
- Nested products with bispinor indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for gamma matrix simplification
```

```
# Arguments
- `self_`: expression containing factorized terms with bispinor indices

# Returns
Expanded expression with factorizations unfolded.

def expand_bis(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import expand_bis
help(expand_bis)
```

Exhaustive reference · Source

`expand_color`

```
Expands factorized terms containing SU(N) color indices.

Expands factorized terms containing SU(N) color indices.

Finds and expands factorized expressions involving color tensors and fields,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

**Factorization Expansion:**
- `q(a) * (T(b,a,c) + S(b,a,c)) → q(a)*T(b,a,c) + q(a)*S(b,a,c)`
- Nested products with color indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for color algebra simplification

**Applications:**
- Expanding factorized QCD expressions before simplification
- Preparing for SU(N) algebra algorithms
- Unfolding nested products in gauge theory calculations
- Algebraic manipulation of color structures

# Arguments
- `self_`: expression containing factorized terms with color indices

# Returns
Expanded expression with factorizations unfolded.

def expand_color(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import expand_color
help(expand_color)
```

Exhaustive reference · Source

`expand_metrics`

```
Expands factorized terms containing metric tensors.

Expands factorized terms containing metric tensors.

Finds and expands factorized expressions involving metric tensors and related
geometric objects, unfolding multiplicative structures for subsequent simplification.
```

```
# Arguments
- `self_`: expression containing factorized metric terms

# Returns
The expanded expression with metric factorizations unfolded.

def expand_metrics(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import expand_metrics
help(expand_metrics)
```

Exhaustive reference · Source

expand_mink

```
Expands factorized terms containing Minkowski spacetime indices.

Expands factorized terms containing Minkowski spacetime indices.

Finds and expands factorized expressions involving Minkowski tensors and vectors,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

**Factorization Expansion:**
- `A(μ) * (B(ν) + C(ν)) → A(μ)*B(ν) + A(μ)*C(ν)`
- Nested products with Minkowski indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for metric simplification and contractions

**Applications:**
- Expanding factorized tensor expressions before simplification
- Preparing for index contraction algorithms
- Unfolding nested products in field theory calculations
- Algebraic manipulation of relativistic expressions

# Arguments
- `self_`: expression containing factorized terms with Minkowski indices

# Returns
Expanded expression with factorizations unfolded.

# Examples:
```python
import symbolica as sp
from symbolica.community.idenso import expand_mink

# Expand factorized vector expression
p = sp.S('p')
q = sp.S('q')
r = sp.S('r')
mu = sp.S('mu')
factorized = p(mu) * (q(mu) + r(mu))
expanded = expand_mink(factorized)  # p(mu)*q(mu) + p(mu)*r(mu)

# Complex factorization
```
```

```
A = sp.S('A')
expr = A * (p(mu) * q(mu) + r(mu))
result = expand_mink(expr)  # A*p(mu)*q(mu) + A*r(mu)
```

```
def expand_mink(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import expand_mink
help(expand_mink)
```

Exhaustive reference · Source

`expand_mink_bis`

```
Expands factorized terms containing both Minkowski and bispinor indices.
```


```
Expands factorized terms containing both Minkowski and bispinor indices.
```


```
Combines the functionality of `expand_mink()` and `expand_bis()` to perform
simultaneous expansion of factorized expressions involving both spacetime
and spinor indices.

# Arguments
- `self_`: expression containing factorized terms with both index types

# Returns
Expanded expression with all factorizations unfolded.
```

```
def expand_mink_bis(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import expand_mink_bis
help(expand_mink_bis)
```

Exhaustive reference · Source

`initialize`

```
Register Idenso's built-in representations and algebra symbols with Symbolica.
```


```
Register Idenso's built-in representations and algebra symbols with Symbolica.
```


```
Symbolica calls this during community-module initialization. Calling it again
is safe and ensures the standard Lorentz, spinor, and color objects exist.
```

```
def initialize() -> None:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import initialize
help(initialize)
```

Exhaustive reference · Source

`list_dangling`

```
Lists the dangling (external, uncontracted) indices present in the expression.
```

```
Lists the dangling (external, uncontracted) indices present in the expression.

Identifies and returns all indices that are not summed over (i.e., not dummy
indices). These are the "free" indices that appear in the final result and
determine the tensor rank of the expression. For dualizable representations,
downstairs indices are represented wrapped in `dind(...)`.

This is essential for:
- Verifying index conservation in tensor equations
- Determining the rank and structure of tensor expressions
- Debugging index contractions

# Arguments
- `self_`: tensor expression to analyze

# Returns
A list of expressions, each representing a free (dangling) index.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import (
list_dangling,
)

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, nu, x)  # T(mu, nu; x)
# print(tensor_with_args)
print(list_dangling(tensor_with_args.to_expression()))
```
def list_dangling(self_: Expression) -> builtins.list[Expression]:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import list_dangling
help(list_dangling)
```

Exhaustive reference · Source

`simplify_color`

```
Applies SU(N) color algebra rules to simplify color group structures.
```

```
Applies SU(N) color algebra rules to simplify color group structures.

Performs comprehensive simplifications of SU(N) color algebra including:

**Structure constants:**
- `f^{abc}f^{ade} = CA δ^{bc}` (Casimir relations)
- `f^{abc}f^{bcd} = CA f^{acd}` (Jacobi identities)
- Antisymmetry: `f^{abc} = -f^{bac}`
```

```
**Generators and traces:**
- `Tr(T^a T^b) = TR δ^{ab}` (orthogonality)
- `T^a_{ij} T^a_{kl} = δ_{il}δ_{jk}/Nc - δ_{ij}δ_{kl}/Nc²` (Fierz identity)
- `(T^a)² = CF` (fundamental Casimir)

**Color factors:**
- `Nc`: Number of colors
- `CA = Nc`: Adjoint Casimir
- `CF = (Nc² - 1)/(2Nc)`: Fundamental Casimir
- `TR = 1/2`: Normalization factor

# Arguments
- `self_`: expression containing SU(N) color structures

# Returns
Simplified expression with color algebra reduced to scalar factors (`Nc`, `CA`, `CF`,
`TR`) when possible.

# Notes
If explicit color indices remain after simplification, it indicates the expression
could not be fully reduced to color-scalar form.
```

```
def simplify_color(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import simplify_color
help(simplify_color)
```

Exhaustive reference · Source

`simplify_gamma`

```
Applies Clifford algebra rules and trace identities to simplify gamma matrix
expressions.
```

```
Applies Clifford algebra rules and trace identities to simplify gamma matrix
expressions.

Performs comprehensive simplifications of Dirac gamma matrix algebra including:
- **Anticommutation relations**: `{γᵘ, γᵛ} = 2gᵘᵛ`
- **Trace identities**: `Tr(γᵘ) = 0`, `Tr(γᵘγᵛ) = 4gᵘᵛ`, etc.
- **Gamma5 properties**: `{γ₅, γᵘ} = 0`, `(γ₅)² = 1`
- **Chain simplifications**: Reduces products of gamma matrices
- **Contraction rules**: Simplifies contracted gamma matrix products

The function recognizes gamma matrices represented as
`spenso::gamma(spenso::mink(dim,mu), spenso::bis(dim,alpha), spenso::bis(dim,beta))`
where `mu` is the Lorentz index and `alpha`, `beta` are spinor indices.
These can be easily created using the hep_lib.

# Arguments
- `self_`: expression containing gamma matrix products and traces

# Returns
The simplified expression with gamma algebra applied.
```

```
# Examples:
```python
from symbolica.community.spenso import TensorLibrary, TensorName
from symbolica.community.idenso import simplify_gamma
from symbolica import S, Expression
# Get HEP library with standard tensors
hep_lib = TensorLibrary.hep_lib()
# Access standard tensors like gamma matrices
gamma_structure = hep_lib[S("spenso::gamma")]
print(gamma_structure)
print(simplify_gamma(gamma_structure(7, 3, 4) * gamma_structure(3, 7, 4)))
```

def simplify_gamma(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import simplify_gamma
help(simplify_gamma)
```

Exhaustive reference · Source

`simplify_metrics`

```
Simplifies contractions involving metric tensors and identity tensors.
```

Simplifies contractions involving metric tensors and identity tensors.

Applies fundamental tensor algebra rules for metric and identity tensors:

Metric tensor rules:
- `gᵘᵛ pᵥ → pᵘ` (index raising/lowering)
- `gᵘᵛ gᵥρ → gᵘρ` or `δᵘρ` (metric composition)
- `gᵘᵤ → D` (dimension of spacetime)
- `ηᵘᵛ pᵥ → pᵘ` (flat metric contractions)

Identity tensor rules:
- `δᵘᵛ pᵥ → pᵘ` (Kronecker delta contraction)
- `δᵘᵤ → D` (trace of identity)

The function recognizes metrics as `spenso::g(...)`

Arguments
- `self_`: expression containing metric/identity tensor contractions

Returns
The simplified expression with metric rules applied.

Examples:
```python
from symbolica.community.idenso import simplify_metrics, to_dots
from symbolica.community.spenso import Representation, TensorName
q = TensorName("q")
g = TensorName.g()
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
```

```
print(simplify_metrics(g(mu, nu) * q(mu)))
```

```
def simplify_metrics(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import simplify_metrics
help(simplify_metrics)
```

Exhaustive reference · Source

`to_dots`

Converts contracted Lorentz/Minkowski indices into dot product notation.

Converts contracted Lorentz/Minkowski indices into dot product notation.

Automatically identifies and converts patterns like `p(mink(D, mu)) * q(mink(D, mu))` into the compact, representation-carrying `dot(p(mink(D)), q(mink(D)))` notation. This
simplification is essential for physics calculations involving four-vectors.

The function recognizes:
- Contracted vector indices: `pᵘqᵤ → p·q`
- Multiple contractions: `pᵘqᵤrᵛsᵥ → (p·q)(r·s)`
- Self-contractions: `pᵘpᵤ → p²`

Arguments
- `self_`: expression containing contracted Minkowski vector indices

Returns
The expression with vector contractions converted to dot products.

Examples:
```python
from symbolica.community.idenso import to_dots
from symbolica.community.spenso import Representation, TensorName
p = TensorName("p")
q = TensorName("q")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")

print(to_dots( p(mu)*q(mu)))
```

```
def to_dots(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import to_dots
help(to_dots)
```

Exhaustive reference · Source

wrap_dummies

Wraps only the dummy (contracted) indices within the expression using a header symbol.

Wraps only the dummy (contracted) indices within the expression using a header symbol.

```
Similar to `wrap_indices`, but selectively identifies and wraps only contracted
indices (those appearing once upstairs and once downstairs, or twice in a
self-dual representation), leaving external (dangling) indices untouched.
This is crucial for proper index management in tensor calculations.

Contracted indices are those that:
- Appear in both upper and lower positions (for dualizable reps)
- Appear twice in the same position (for self-dual reps)
- Are summed over (Einstein summation convention)

# Arguments
- `self_`: input expression containing both dummy and free indices
- `header`: symbol to use as wrapper function name for dummy indices only

# Returns
A new expression with only contracted indices wrapped.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import simplify_metrics, wrap_dummies

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, nu, x)  # T(mu, nu; x)
# print(tensor_with_args)
print(wrap_dummies(tensor_with_args.to_expression(), sp.S("wrap")))

```

def wrap_dummies(self_: Expression, header: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```
from symbolica.community.idenso import wrap_dummies
help(wrap_dummies)
```

Exhaustive reference · Source

wrap_indices

Wrap all abstract indices with a header symbol # Arguments - `self_`: input expression containing tensor indices - `header`: symbol to use as the wrapper function for all indices # Returns Expression with all indices wrapped by the header symbol.

```
Wrap all abstract indices with a header symbol

# Arguments
- `self_`: input expression containing tensor indices
- `header`: symbol to use as the wrapper function for all indices

# Returns
Expression with all indices wrapped by the header symbol.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import wrap_indices

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, x)  # T(mu, nu; x)
print(tensor_with_args)
print(wrap_indices(tensor_with_args.to_expression(), sp.S("wrap")))

```

def wrap_indices(self_: Expression, header: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Inspect the registered export

```python
from symbolica.community.idenso import wrap_indices
help(wrap_indices)
```

Exhaustive reference · Source

Canonical package changelog

The following release history is imported as typed Markdown from `crates/idenso/CHANGELOG.md`.

Changelog

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

[Unreleased]

0.2.5 - 2026-01-08

Fixed

- fix metric initialization and update to symbolica 1.2
- fix initialize to also load ETS

Other

- update linnet

0.2.4 - 2025-12-15

Fixed

- fix canonization in presence of duals
- fix gamma normalisation
- fix warnings
- fix add_assign with sparse tensors
- fix projp

Other

- Import initialize function
- Update symbolica to 1.1.0
- update linnet
- align to atom in argument of cof for Nc
- update linnet
- add readmes
- formatting and add ref for parametric
- update to 0.17 pyo3_stub_gen
- update linnet and make classattr to classmethods
- proper imports and feature flags for python
- move initialize to api
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- update to current dev symbolica and add pattern for pslash + m conjugate
- update to linnet 0.13.1
- working canonize (up to functions) and dirac adjoint
- spenso canonize buggy
- robust_conj but with lots of gamma_0s
- first try conj

- working pow execution with wrapping
- add function generic key

0.2.2 - 2025-08-18

Fixed
- fix cook indices
- fix clippy errors

Other
- update linnet to crates
- all tests pass
- Fix Multiple Heads Bug
- update linnet
- update linnet
- update linnet
- update parse problem and add spenso:: to all symbols
- make all symbols take spenso namespace
- operate directly on coefficient list
- remove expand

0.2.1 - 2025-07-15

Fixed
- fix gamma algebra again

Other
- update versions

0.2.0 - 2025-07-15

Fixed
- fix wrap_indices
- fix gamma algebra≈

Other
- allow arbitrary arguments for to dots

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/idenso/latest/`
- **Source revision:** `dfddcec2a39e23d2d5faba16c18f197dc3301473`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python_api, python_abi, python_stubgen`)
- `linnet/linnet` — `0.17.0` (`features: nodestore-vec, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3-py310`)
- `spenso/spenso` — `0.6.0` (`features: shadowing`)
- `spenso/spenso-macros` — `0.3.2` (`features: shadowing`)
- `spenso/spenso-hep-lib` — `0.1.7` (`no optional features`)
- `spenso/spynso3` — `0.1.2` (`features: python_stubgen`)
- `idenso/idenso` — `0.3.0` (`features: bincode, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python_stubgen`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: symbolica_community_module, python_stubgen`)